

Связанные данные и запросы в Prisma

Работа с отношениями между моделями через `include`, `select` и `relation filters`

Результаты занятия

- ✓ Понимать, как отношения PostgreSQL отражаются в Prisma Schema
- ✓ Различать scalar fields и relation fields
- ✓ Получать связанные модели через include
- ✓ Ограничивать связанные данные через вложенный select
- ✓ Получать Product вместе с Category
- ✓ Получать Order вместе с User, OrderItem и Product
- ✓ Фильтровать родительские записи по связанным данным
- ✓ Использовать some, every и none для связей один-ко-многим
- ✓ Формировать удобный вложенный JSON для REST API

Учебные вопросы

- 1 Связи в Prisma Schema** Как PK и FK превращаются в relation fields
- 2 include и select** Как получать связанные данные и ограничивать результат
- 3 Product ↔ Category** Практический запрос связи многие-к-одному
- 4 Order ↔ OrderItem ↔ Product** Получение многоуровневой структуры данных
- 5 Фильтрация по связям** some, every, none и применение в Service



Учебный вопрос 1. Связи в Prisma Schema

Relation fields описывают связи между моделями

Зачем нужны связанные данные

- В интернет-магазине данные разделены между несколькими таблицами.
- Product хранит categoryId, но название категории находится в Category.
- Order хранит userId, но данные пользователя находятся в User.
- Позиции заказа находятся в OrderItem, а сведения о товаре — в Product.
- REST API часто должен вернуть эти данные одним вложенным JSON.
- Связанные запросы позволяют получить логически целый объект из нескольких таблиц.

PostgreSQL и Prisma описывают одну связь по-разному

PostgreSQL

- `products.category_id` — внешний ключ
- `orders.user_id` — внешний ключ
- `order_items.order_id` — внешний ключ
- `order_items.product_id` — внешний ключ

Prisma

- `Product.category` — relation field
- `Category.products` — relation field
- `Order.user` — relation field
- `Order.items` — relation field

Category ↔ Product в Prisma Schema

PRISMA

```
model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique
  products Product[]

  @@map("categories")
}

model Product {
  id      Int      @id @default(autoincrement())
  name    String
  price   Int
  categoryId Int    @map("category_id")
  category Category @relation(fields: [categoryId], references: [id])

  @@map("products")
}
```

Что важно

- categoryId — обычное scalar field, которое хранится в таблице products.
- category — relation field для перехода от Product к Category.
- products — relation field-массив для перехода от Category к Product[].
- @relation связывает categoryId с Category.id.

Как внешний ключ превращается в relation field Prisma

Одна и та же связь: в базе данных — внешний ключ, в Prisma — удобные relation fields.



PostgreSQL

Таблицы и внешний ключ

FK

categories

id	name
1	Электроника
2	Одежда
3	Книги
...	...

products

id	name	category_id
1	Ноутбук	1
2	Футболка	2
3	Книга	3
...	...	...



category_id хранится в таблице **products**

Одна связь
в двух
представлениях

FK

Внешний
ключ

Внешний ключ
в базе данных
становится
relation fields
в Prisma.



Prisma

Модели и relation fields

model Product {

```
  id      Int      @id
  name    String
  categoryId Int
  // хранимый скалярный field
  category Category @relation(...)
  // relation field (ссылка на Category)
}
```

model Category {

```
  id      Int      @id
  name    String
  products Product[]
  // relation field (много Product)
}
```

Product.category
(ссылка на Category)

Category.products
(много Product)



Важно: **categoryId** хранится в таблице,
а **category** существует на уровне Prisma-модели.

Scalar field и relation field

Scalar field

- Хранится как столбец таблицы
- Примеры: id, name, price, categoryId
- Возвращается обычным запросом Prisma
- Имеет тип Int, String, Boolean и т.д.

Relation field

- Описывает связь между моделями
- Примеры: category, products, items
- Не является отдельным столбцом таблицы
- Загружается явно через include или select

Три связи нашего Internet Shop

● Category → Product

1:N. Одна категория
содержит много товаров.

● User → Order

1:N. Один пользователь
может иметь много заказов.

● Order ↔ Product

M:N реализуется через
промежуточную модель
OrderItem.

Order и OrderItem

PRISMA

```
model Order {
  id          Int          @id @default(autoincrement())
  userId      Int          @map("user_id")
  status      String
  createdAt   DateTime     @default(now()) @map("created_at")
  user        User         @relation(fields: [userId], references: [id])
  items       OrderItem[]

  @@map("orders")
}

model OrderItem {
  orderId      Int          @map("order_id")
  productId    Int          @map("product_id")
  quantity     Int
  price        Int
  order        Order       @relation(fields: [orderId], references: [id])
  product      Product     @relation(fields: [productId], references: [id])

  @@id([orderId, productId])
  @@map("order_items")
}
```

Что важно

- Order.items — список позиций заказа.
- OrderItem.order — ссылка на родительский заказ.
- OrderItem.product — ссылка на товар.
- Составной @@id([orderId, productId]) задаёт составной первичный ключ.



Учебный вопрос 2. include и select

Связи не загружаются автоматически

Что вернёт обычный findMany()

JAVASCRIPT

```
const products = await prisma.product.findMany();
```

Что важно

- По умолчанию Prisma Client возвращает scalar fields модели.
- Relation fields не добавляются автоматически.
- У Product будет categoryId, но объекта category не будет.
- Это позволяет не загружать ненужные связанные данные.
- Связь существует в Prisma Schema, но её данные нужно запросить явно.

Product без include

Запрос: `prisma.product.findUnique({ where: { id: 1 } })`

JAVASCRIPT

```
const product = await
prisma.product.findUnique({
  where: { id: 1 }
});
```

Результат

```
{
  "id": 1,
  "name": "Laptop",
  "price": 420000,
  "categoryId": 2,
  "inStock": true
}
```

Пояснение

- categoryId присутствует как scalar field, но связанного объекта Category в результате нет.

include: добавить связанную модель

JAVASCRIPT

```
const product = await prisma.product.findUnique({  
  where: { id: 1 },  
  include: {  
    category: true  
  }  
});
```

Что важно

- include указывает Prisma Client, какие relation fields нужно добавить к результату.
- Теперь результат содержит Product и вложенный объект category.

Результат include

Запрос: Product вместе с Category

JAVASCRIPT

```
include: {  
  category: true  
}
```

Результат

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 420000,  
  "categoryId": 2,  
  "category": {  
    "id": 2,  
    "name": "Ноутбуки"  
  }  
}
```

Пояснение

- Relation field category становится вложенным объектом JSON.

Запрашивай
только то,
что нужно!

Обычный запрос vs include vs select

Prisma может возвращать разный уровень детализации данных



Prisma
One query.
More possibilities.

Обычный запрос

только scalar fields

TS

```
const product = await prisma.product.findUnique({  
  where: { id: 1 }  
})
```



Результат

JSON

```
{  
  "id": 1,  
  "name": "Ноутбук",  
  "price": 99900,  
  "categoryId": 2,  
  "inStock": true  
}
```



Product

Возвращаются только обычные поля модели (scalar fields).

include

Product + полный Category

TS

```
const product = await prisma.product.findUnique({  
  where: { id: 1 },  
  include: {  
    category: true  
  }  
})
```



Результат

JSON

```
{  
  "id": 1,  
  "name": "Ноутбук",  
  "price": 99900,  
  "categoryId": 2,  
  "inStock": true,  
  "category": {  
    "id": 2,  
    "name": "Электроника",  
    "description": "Техника и гаджеты",  
    "createdAt": "2024-01-10T12:00:00.000Z"  
  }  
}
```

Полный объект
Category



Product

+



Category

Возвращается Product и связанная модель Category со всеми её полями.

вложенный select

только выбранные поля Product и Category

TS

```
const product = await prisma.product.findUnique({  
  where: { id: 1 },  
  select: {  
    id: true,  
    name: true,  
    price: true,  
    category: {  
      select: {  
        id: true,  
        name: true  
      }  
    }  
  }  
})
```



Результат

JSON

```
{  
  "id": 1,  
  "name": "Ноутбук",  
  "price": 99900,  
  "category": {  
    "id": 2,  
    "name": "Электроника"  
  }  
}
```

Только выбранные
поля Product

Только выбранные
поля Category



Product

+



Category

Возвращаются только указанные поля из Product и Category.



include — удобно, когда нужны все данные связанной модели.

select — когда важен контроль над структурой и объёмом ответа.

select: вернуть только нужные поля

JAVASCRIPT

```
const product = await prisma.product.findUnique({
  where: { id: 1 },
  select: {
    id: true,
    name: true,
    price: true
  }
});
```

Что важно

- select формирует точную структуру результата и позволяет выбрать как scalar fields, так и relation fields.

Вложенный select для Category

JAVASCRIPT

```
const products = await prisma.product.findMany({
  select: {
    id: true,
    name: true,
    price: true,
    category: {
      select: {
        id: true,
        name: true
      }
    }
  }
});
```

Что важно

- Верхний select выбирает поля Product.
- category открывает связанную модель.
- Вложенный select выбирает только id и name категории.
- API не получает лишние поля.

include или select?

include

- Добавляет связь к обычному набору scalar fields
- Удобен для быстрого получения всей связанной модели
- Простой синтаксис
- Подходит для учебных и внутренних API

select

- Явно задаёт структуру результата
- Можно выбирать конкретные поля связи
- Снижает объём ответа
- Удобен для публичного REST API

Практический принцип для REST API

- Не возвращайте связанные данные только потому, что они доступны.
- Frontend должен получать данные, необходимые конкретному экрану.
- Для карточки товара достаточно Category.id и Category.name.
- Для административной страницы может потребоваться больше полей.
- Структура Prisma-запроса должна соответствовать структуре API-ответа.
- Чем точнее select, тем понятнее контракт REST API.



Учебный вопрос 3. Product ↔ Category

Первый практический связанный запрос

Список товаров с категориями

JAVASCRIPT

```
export async function getAllProducts() {  
  return prisma.product.findMany({  
    include: {  
      category: true  
    },  
    orderBy: {  
      id: "asc"  
    }  
  });  
}
```

Что важно

- `findMany()` получает коллекцию `Product`.
- `include.category` добавляет `Category` к каждому `Product`.
- `orderBy` сортирует товары.
- `Service` возвращает готовый массив JavaScript-объектов.

JSON списка товаров

Запрос: GET /api/products

JAVASCRIPT

```
include: {  
  category: true  
}
```

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 420000,  
    "categoryId": 2,  
    "category": {  
      "id": 2,  
      "name": "Ноутбуки"  
    }  
  }  
]
```

Пояснение

- Каждый элемент массива получает вложенную Category.

Компактный Product API через select

JAVASCRIPT

```
return prisma.product.findMany({
  select: {
    id: true,
    name: true,
    price: true,
    inStock: true,
    category: {
      select: {
        id: true,
        name: true
      }
    }
  },
  orderBy: { id: "asc" }
});
```

Что важно

- categoryId можно не возвращать, если frontend получает category.id.
- В ответе остаются только поля, необходимые клиенту.
- Связанная Category представлена компактным объектом.

Product + Category: от двух таблиц к одному JSON

Связанные данные → Prisma → один вложенный JSON

Better
Developers
Brighter
Tomorrow



База данных

Две связанные таблицы



products

id	name	price	category_id
1	Laptop	420000	2



FK category_id

products.category_id
ссылается на categories.id



categories

id	name
2	Ноутбуки



Prisma include / select

Объединяет связанные данные

Вариант 1: include (проще)

```
const products = await prisma.product.findMany({
  include: {
    category: true
  }
})
```

или

Вариант 2: select (больше контроля)

```
const products = await prisma.product.findMany({
  select: {
    id: true,
    name: true,
    price: true,
    category: {
      select: {
        id: true,
        name: true
      }
    }
  }
})
```



Результат: один JSON

Данные из Product и Category в одном ответе

```
{
  "id": 1,
  "name": "Laptop",
  "price": 420000,
  "category": {
    "id": 2,
    "name": "Ноутбуки"
  }
}
```

Product и Category
объединены в один
JSON-ответ!



Prisma превращает данные из связанных таблиц в один удобный JSON-ответ.



Больше,
чем просто БД.
Больше возможностей!

Фильтрация Product по Category

JAVASCRIPT

```
const products = await prisma.product.findMany({
  where: {
    category: {
      is: {
        name: "Ноутбуки"
      }
    }
  },
  include: {
    category: true
  }
});
```

Что важно

- where может обращаться к связанным моделям.
- category — relation field Product.
- is задаёт условие для связи многие-к-одному.
- Возвращаются Product, категория которых соответствует условию.

Мини-практика: Product и Category

Измените Product Service так, чтобы GET /api/products возвращал название категории каждого товара.

Задание

- Сначала реализуйте вариант через include
- Проверьте структуру JSON
- Затем замените include на вложенный select
- Оставьте в Category только id и name

Критерии проверки

- В ответе присутствует связанная категория
- Нет ручного SQL JOIN
- Prisma-запрос расположен в Service
- JSON подходит для frontend



Учебный вопрос 4. Order ↔ OrderItem ↔ Product

Многоуровневая вложенная структура

Какие данные нужны странице заказа

- Сам заказ: `id`, `status`, `createdAt`.
- Покупатель: `User`.
- Позиции заказа: `OrderItem[]`.
- Для каждой позиции: `quantity` и зафиксированная `price`.
- Для каждой позиции: `Product.id` и `Product.name`.
- Дополнительно API может вычислить общую сумму заказа.

Order вместе с User и OrderItem

JAVASCRIPT

```
const order = await prisma.order.findUnique({
  where: { id: 10 },
  include: {
    user: true,
    items: true
  }
});
```

Что важно

- user — relation field типа User.
- items — relation field типа OrderItem[].
- В результате получаем вложенный объект и вложенный массив.
- Product внутри OrderItem пока ещё не загружен.

Вложенный include: OrderItem → Product

JAVASCRIPT

```
const order = await prisma.order.findUnique({
  where: { id: 10 },
  include: {
    user: true,
    items: {
      include: {
        product: true
      }
    }
  }
});
```

Что важно

- items содержит настройки запроса связанных OrderItem.
- Внутри items можно снова использовать include.
- product добавляется к каждой позиции заказа.
- Один Prisma-запрос описывает всю требуемую структуру.

Точный Order API через вложенный select

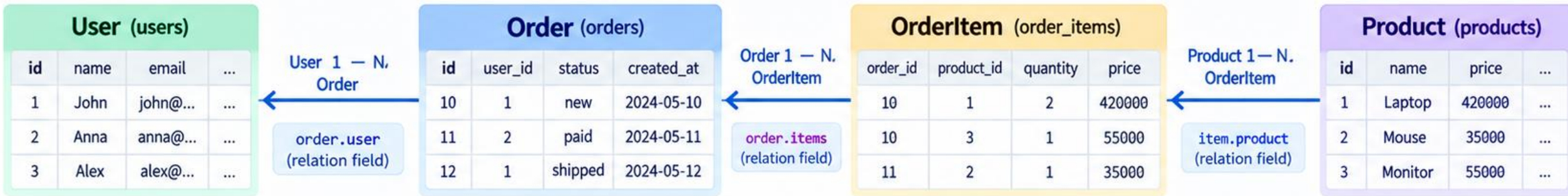
JAVASCRIPT

```
const order = await prisma.order.findUnique({
  where: { id },
  include: {
    user: {
      select: {
        id: true,
        name: true,
        email: true
      }
    },
    items: {
      include: {
        product: {
          select: {
            id: true,
            name: true
          }
        }
      }
    }
  }
});
```

Что важно

- User ограничен тремя полями.
- Product внутри каждой позиции ограничен id и name.
- quantity и price берутся из OrderItem.
- Полученная структура готова для REST-ответа.

Многоуровневые связанные данные через вложенный include



Prisma-запрос с вложенным include

```
1 const order = await prisma.order.findUnique({
2   where: { id: 10 },
3   include: {
4     user: true,
5     items: {
6       include: {
7         product: true
8       }
9     }
10  }
11 });
```

Один запрос —
три связанные модели

1

include.user —
загружает пользователя
заказа

2

include.items —
загружает позиции заказа

3

include.items.include.product —
загружает товар
для каждой позицииPrisma автоматически выполняет
необходимые JOIN и возвращает
вложенный JavaScript-объект.

Результат (вложенный JSON)

```
1 {
2   "id": 10,
3   "status": "new",
4   "createdAt": "2024-05-10T10:00:00.000Z",
5   "user": {
6     "id": 1,
7     "name": "John",
8     "email": "john@example.com"
9   },
10  "items": [
11    {
12      "productId": 1,
13      "quantity": 2,
14      "price": 420000,
15      "product": {
16        "id": 1,
17        "name": "Laptop"
18      }
19    }
20  ]
21 }
```

Данные пользователя (User)

Массив позиций заказа (OrderItem[])

Данные товара (Product) в каждой позиции



Вложенный include позволяет получить сложную структуру данных одним Prisma-запросом без ручного SQL.

Вложенный JSON заказа

Запрос: GET /api/orders/10

JAVASCRIPT

```
findUnique({ include: { user, items: {  
  include: { product } } } })
```

Результат • JSON

```
{  
  "id": 10,  
  "status": "new",  
  "user": {  
    "id": 3,  
    "name": "Alex"  
  },  
  "items": [  
    {  
      "productId": 1,  
      "quantity": 2,  
      "price": 420000,  
      "product": {  
        "id": 1,  
        "name": "Laptop"  
      }  
    }  
  ]  
}
```

Пояснение

- REST API может вернуть данные нескольких связанных таблиц как один логический ресурс Order.

Цена Product и цена OrderItem — не одно и то же

- `Product.price` — текущая цена товара в каталоге.
- `OrderItem.price` — цена, зафиксированная при оформлении заказа.
- `Product.price` может измениться позже.
- История заказа должна сохранять исходную стоимость.
- Сумму заказа следует рассчитывать по `OrderItem.price × quantity`.
- Для истории заказа используем `OrderItem.price`, а не текущий `Product.price`.

Расчёт общей суммы заказа

JAVASCRIPT

```
const total = order.items.reduce(
  (sum, item) => sum + item.price * item.quantity,
  0
);

return {
  ...order,
  total
};
```

Что важно

- Prisma возвращает связанные данные.
- Service может добавить бизнес-логику после запроса.
- `reduce()` суммирует стоимость всех позиций.
- `total` не хранится в таблице, а вычисляется приложением.



Учебный вопрос 5. Фильтрация по СВЯЗЯМ

Ищем родительские записи по условиям дочерних моделей

Relation filters для коллекций

- Для связи один-ко-многим Prisma позволяет фильтровать родительские записи по связанным элементам через `some`, `every` и `none`.
- `some` — существует хотя бы одна подходящая связанная запись.
- `every` — все связанные записи соответствуют условию.
- `none` — ни одна связанная запись не соответствует условию.

Заказы, содержащие определённый Product

JAVASCRIPT

```
const orders = await prisma.order.findMany({
  where: {
    items: {
      some: {
        productId: 1
      }
    }
  }
});
```

Что важно

- Order.items — связь один-ко-многим.
- some проверяет наличие хотя бы одной позиции.
- productId: 1 задаёт условие для OrderItem.
- Результат содержит только заказы, где встречается Product с id = 1.

some, every, none

Оператор	Смысл	Пример
some	Хотя бы одна связанная запись подходит	Заказы, где есть Product 1
every	Все связанные записи подходят	Заказы, где все позиции удовлетворяют условию
none	Ни одна связанная запись не подходит	Заказы без указанного Product

Категории, где есть товары в наличии

JAVASCRIPT

```
const categories = await prisma.category.findMany({
  where: {
    products: {
      some: {
        inStock: true
      }
    }
  },
  include: {
    products: true
  }
});
```

Что важно

- Category.products — relation field-массив.
- some проверяет наличие хотя бы одного Product.
- inStock: true — условие для связанных товаров.
- include возвращает связанные Products вместе с Category.

Находим все заказы, в которых есть хотя бы одна позиция с указанным товаром

Товары (products)

id	name	price	...
1	Laptop	420000	...
2	Mouse	35000	...
3	Monitor	55000	...



Ищем заказы,
в которых есть товар
с id = 1 (Laptop)

Prisma-запрос

```
const orders = await prisma.order.findMany({
  where: {
    items: {
      some: {
        productId: 1
      }
    }
  },
  include: {
    items: {
      include: {
        product: true
      }
    }
  },
  orderBy: { id: "asc" }
});
```

some — вернуть Order,
если хотя бы одна
связанная запись OrderItem
удовлетворяет условию
(productId = 1)



some используется для связей «один ко многим». Он проверяет наличие хотя бы одной подходящей связанной записи.

Примеры заказов и результат фильтрации

Заказ	Позиции заказа (order_items)	Результат									
Order #10 id: 10 user_id: 1 status: new	<table><tr><th>product_id</th><th>quantity</th><th>price</th></tr><tr><td>1</td><td>2</td><td>420000</td></tr><tr><td>2</td><td>1</td><td>35000</td></tr></table>	product_id	quantity	price	1	2	420000	2	1	35000	Включается (есть productId = 1)
product_id	quantity	price									
1	2	420000									
2	1	35000									
Order #11 id: 11 user_id: 2 status: paid	<table><tr><th>product_id</th><th>quantity</th><th>price</th></tr><tr><td>2</td><td>1</td><td>35000</td></tr><tr><td>3</td><td>1</td><td>55000</td></tr></table>	product_id	quantity	price	2	1	35000	3	1	55000	Исключается (нет productId = 1)
product_id	quantity	price									
2	1	35000									
3	1	55000									
Order #12 id: 12 user_id: 1 status: shipped	<table><tr><th>product_id</th><th>quantity</th><th>price</th></tr><tr><td>3</td><td>1</td><td>55000</td></tr><tr><td>1</td><td>1</td><td>420000</td></tr></table>	product_id	quantity	price	3	1	55000	1	1	420000	Включается (есть productId = 1)
product_id	quantity	price									
3	1	55000									
1	1	420000									
Order #13 id: 13 user_id: 3 status: new	<table><tr><th>product_id</th><th>quantity</th><th>price</th></tr><tr><td>2</td><td>3</td><td>35000</td></tr><tr><td>3</td><td>1</td><td>55000</td></tr></table>	product_id	quantity	price	2	3	35000	3	1	55000	Исключается (нет productId = 1)
product_id	quantity	price									
2	3	35000									
3	1	55000									



Relation filter some позволяет находить родительские записи по условиям связанных моделей. В результате вернутся только те заказы, в которых есть хотя бы одна позиция с указанным товаром.

Другие операторы

- every** — все связанные записи соответствуют условию
- none** — ни одна связанная запись не соответствует условию

Связанный запрос остаётся частью Service

- Route определяет URL.
- Controller работает с req, res и HTTP-статусами.
- Service формирует Prisma-запрос и бизнес-логику.
- Prisma Client получает связанные данные.
- PostgreSQL остаётся физическим хранилищем таблиц и связей.
- Controller не должен знать, используется include, select или relation filter.

Order Service целиком

```
export async function getOrderDetails(id) {
  const order = await prisma.order.findUnique({
    where: { id },
    include: {
      user: {
        select: { id: true, name: true, email: true }
      },
      items: {
        include: {
          product: {
            select: { id: true, name: true }
          }
        }
      }
    }
  });

  if (!order) return null;

  const total = order.items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  return { ...order, total };
}
```

Что важно

- `findUnique()` получает Order.
- `include` загружает User и OrderItem[].
- Вложенный `select` ограничивает User и Product.
- Service вычисляет total.
- Controller получает уже подготовленный объект.

Практика: страница заказа Internet Shop

Реализуйте GET /api/orders/:id, который возвращает заказ, пользователя, позиции заказа, данные товаров и общую сумму.

Задание

- Создать Order Service
- Получить User через relation field
- Получить OrderItem[]
- Вложено получить Product
- Ограничить поля через select
- Вычислить total
- Вернуть 404 для отсутствующего заказа

Критерии проверки

- Нет ручного SQL JOIN
- Prisma-запрос находится в Service
- Ответ содержит вложенные данные
- total рассчитан по OrderItem.price
- Controller отвечает только за HTTP

1 HTTP-запрос от клиента

GET /api/orders/10


Frontend
(браузер / приложение)**2 Router**
(маршрутизация)

routes/orderRoutes.js

```
1 router.get(
2  ('/:id',
3   getOrderById
4 );
```

Определяет URL
и передаёт запрос
в Controller.**3 Controller**
(HTTP и обработка)

controllers/orderController.js

```
1 export async function
2   getOrderById(req, res) {
3   const id = Number(
4     req.params.id
5   );
6   const order = await
7     getOrderDetails(id);
8   res.json(order);
9 }
```

Работает с req, res,
обрабатывает ошибки,
возвращает JSON.**4 Order Service**
(бизнес-логика)

services/orderService.js

```
1 export async function
2   getOrderDetails(id) {
3   const order = await
4     prisma.order.findUnique({
5     where: { id },
6     include: {
7       user: true,
8       items: { include: {
9         product: true
10      } } }
11   });
12   if (!order) return null;
13   // Вычисление общей суммы
14   const total = order.items
15     .reduce((sum, item) =>
16       sum + item.price * item.quantity,
17       0
18     );
19   return { ...order, total };
```

 Вычисление total выполняется
в Service после получения данных
из базы.**5 Prisma Client**
(запрос к БД)

prisma.order.findUnique(...)

```
1 prisma.order.findUnique({
2   where: { id: 10 },
3   include: {
4     user: true,
5     items: { include: {
6       product: true
7     } }
8   }
9 })
```

Prisma формирует SQL
запрос с необходимыми
JOIN и получает связанные
данные одним запросом.**6 PostgreSQL**
(база данных)**users**
id, name, email, ...**orders**
id, user_id, status, ...**order_items**
order_id, product_id,
quantity, price, ...**products**
id, name, price, ...База данных возвращает
связанные данные
из нескольких таблиц.**7 Вложенный**
JavaScript-объект

```
{
  id: 10,
  status: "new",
  createdAt: "2024-05-10T10:00:00Z",
  user: {
    id: 1,
    name: "John",
    email: "john@example.com"
  },
  items: [
    {
      productId: 1,
      quantity: 2,
      price: 420000,
      product: { id: 1, name: "Laptop" }
    },
    {
      productId: 3,
      quantity: 1,
      price: 550000,
      product: { id: 3, name: "Monitor" }
    }
  ],
  total: 895000
}
```

Service добавляет поле total
и возвращает готовый объект
в Controller.**8 JSON-ответ**
клиенту

```
{
  "id": 10,
  "status": "new",
  "createdAt": "2024-05-10T10:00:00Z",
  "user": {
    "id": 1,
    "name": "John",
    "email": "john@example.com"
  },
  "items": [
    {
      "productId": 1,
      "quantity": 2,
      "price": 420000,
      "product": {
        "id": 1,
        "name": "Laptop"
      }
    },
    {
      "productId": 3,
      "quantity": 1,
      "price": 550000,
      "product": {
        "id": 3,
        "name": "Monitor"
      }
    }
  ],
  "total": 895000
}
```

 Controller отправляет JSON
клиенту через res.json().**Результат**

- ✓ Один HTTP-запрос возвращает данные из нескольких связанных таблиц.
- ✓ Используется include для загрузки User, Orderitem и Product.
- ✓ Сумма заказа (total) вычисляется в Service.
- ✓ Frontend получает удобный вложенный JSON.

Краткий путь запроса**Главные идеи**

- Связанные данные загружаются через include.
- Весь Prisma-запрос находится в Service.
- Controller не знает деталей include или select.
- Бизнес-логика (например, расчёт total) выполняется после получения данных из базы.
- Результат — один согласованный JSON для REST API.

Контрольные вопросы

1. Чем scalar field отличается от relation field?

2. Почему relation fields не возвращаются обычным findMany() автоматически?

3. Для чего используется include?

4. Для чего используется вложенный select?

5. Как получить Category вместе с Product?

6. Как получить Product внутри каждого OrderItem?

7. Что означает relation filter some?

8. Чем some отличается от none?

9. Почему сумму заказа лучше рассчитывать по OrderItem.price?

10. В каком слое приложения должны находиться связанные Prisma-запросы?

Итоги занятия

1

Prisma Schema описывает отношения между моделями через `relation fields`.

2

`Scalar fields` хранятся в таблицах, `relation fields` описывают связи.

3

Связанные модели не добавляются к результату автоматически.

4

`include` загружает связанную модель вместе с основной.

5

`select` позволяет точно определить возвращаемые поля.

6

Вложенные `include` и `select` позволяют получать многоуровневые структуры.

7

`Order` может быть получен вместе с `User`, `OrderItem` и `Product`.

8

`some`, `every` и `none` фильтруют записи по связанным коллекциям.

9

`Service` остаётся местом Prisma-запросов и бизнес-логики.

10

Prisma формирует связанные JavaScript-объекты без ручного SQL JOIN.